Project 1

AES-128 Crypto Accelerator

RTL Specification

Abiramasundar S EMP 6874

RTL SPECIFICATION

Project 1 — AES-128 Crypto Accelerator with Secure Register Access Control

Field	Detail
Project Engineer	Abiramasundar S
Project Theme	Hardware Security
Design Type	RTL / Hardware Security
Interface	APB4 Slave
Cryptographic Algorithm	AES-128
Clock Domains	APB clock (pclk) – Slow Clock and Crypto clock (crypto_clk) – Fast Clock
Primary Objective	Secure, constant-time AES-128 encryption/decryption accelerator

Table of Contents

Chapter 1 — Introduction

1.1 Overview of AES-128

1.2 AES-128 Parameters

1.3 AES Encryption Round

1.4 AES Decryption Flow

1.5 Project Scope

1.6 Design Objectives

1.7 Top-Level Architecture

Chapter 2 — Signal Description

Chapter 3 — Register Map and Functional Description

3.1 APB Transfer Detection

3.2 APB Register Map

3.3 CONTROL Register

3.4 Key Loading

3.5 Key Security

3.6 Busy / Done Behavior

3.7 Privilege Checking

3.8 STATUS Register

Chapter 4 — Reset Behavior

Chapter 5 — Verification

5.1 Reference Test Vectors (FIPS-197)

5.2 Example APB Transaction Sequence

Chapter 1 — Introduction

1.1 Overview of AES-128

The AES-128 cryptographic core implements the **Advanced Encryption Standard (AES)** with a 128-bit key and a 128-bit data block, in accordance with NIST **FIPS 197**.

The core performs AES-128 encryption on a single 128-bit plaintext block and generates a 128-bit ciphertext block, and performs the corresponding decryption in reverse.

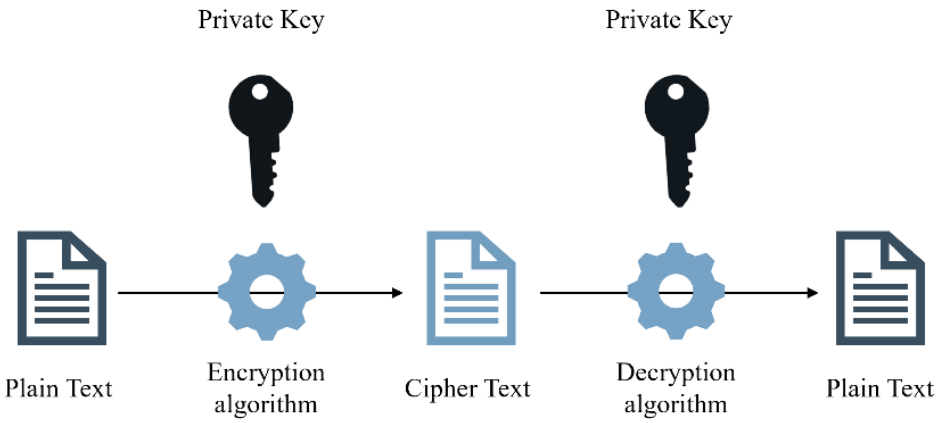


Figure 1.1 — AES encryption/decryption concept

1.2 AES-128 Parameters

Parameter	Specification
Algorithm	AES
Operation	Mode = 1 → Encryption | Mode = 0 → Decryption
Key size	128 bits
Block size	128 bits (per block)
Number of rounds	10 + 1 (Initial round)
Number of round keys	10 + 1
Round-key size	128 bits
Plaintext input	128 bits
Ciphertext output	128 bits

1.3 AES Encryption Round

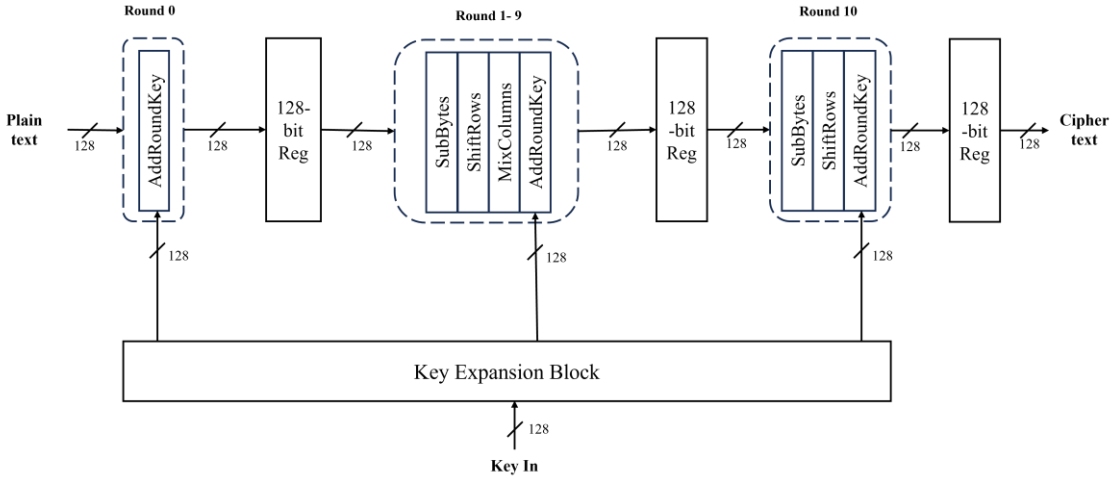


Figure 1.2 — AES encryption round structure

AES-128 encryption consists of 10 transformation rounds, preceded by an initial AddRoundKey operation. Each standard round performs SubBytes, ShiftRows, MixColumns, and AddRoundKey. The final round performs SubBytes, ShiftRows, and AddRoundKey, and does not perform MixColumns.

1.4 AES Decryption Flow

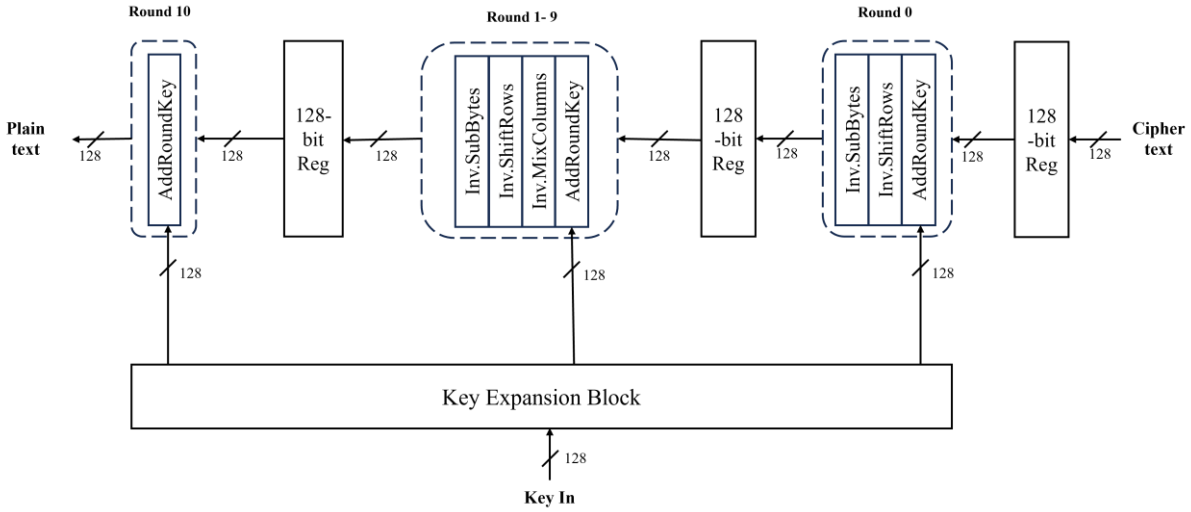


Figure 1.3 — AES decryption round structure

AES-128 decryption likewise consists of 10 rounds, preceded by InvSubBytes, InvShiftRows, and AddRoundKey (this initial stage does not perform InvMixColumns). Each standard round performs InvSubBytes, InvShiftRows, InvMixColumns, and AddRoundKey. The final round performs only AddRoundKey.

1.5 Project Scope

The RTL implementation **supports** the following **features**:

- AES-128 encryption and decryption
- 128-bit plaintext/ciphertext blocks
- 128-bit cryptographic key
- 10 AES rounds
- ECB mode
- APB4 slave interface
- Asynchronous TX and RX FIFOs
- Hardware key locking
- Software-triggered and reset-triggered key zeroization
- Privilege checking using PPROT[0]
- Security-violation detection
- Constant-time AES processing
- FIFO-threshold, security-violation, and error interrupts

The design is restricted to AES-128. The following are outside the mandatory scope:

- AES-192 and AES-256
- GCM and CTR modes
- Full differential-power-analysis (DPA) protection and full fault-injection protection
- Hardware key derivation

1.6 Design Objectives

The primary design objectives are to:

1. Implement a functionally correct AES-128 engine according to FIPS-197.

2. Provide secure configuration and data access through APB4.
3. Prevent software from reading cryptographic key material.
4. Prevent modification of the key after KEY_LOCK is asserted.
5. Provide deterministic, constant-time AES operation.
6. Prevent sensitive intermediate AES states from being externally observable.
7. Safely transfer data between asynchronous clock domains.
8. Provide sufficient verification visibility without exposing sensitive key material.
9. Support synthesis and static timing analysis (STA) for a 200 MHz crypto-clock target.

1.7 Top-Level Architecture

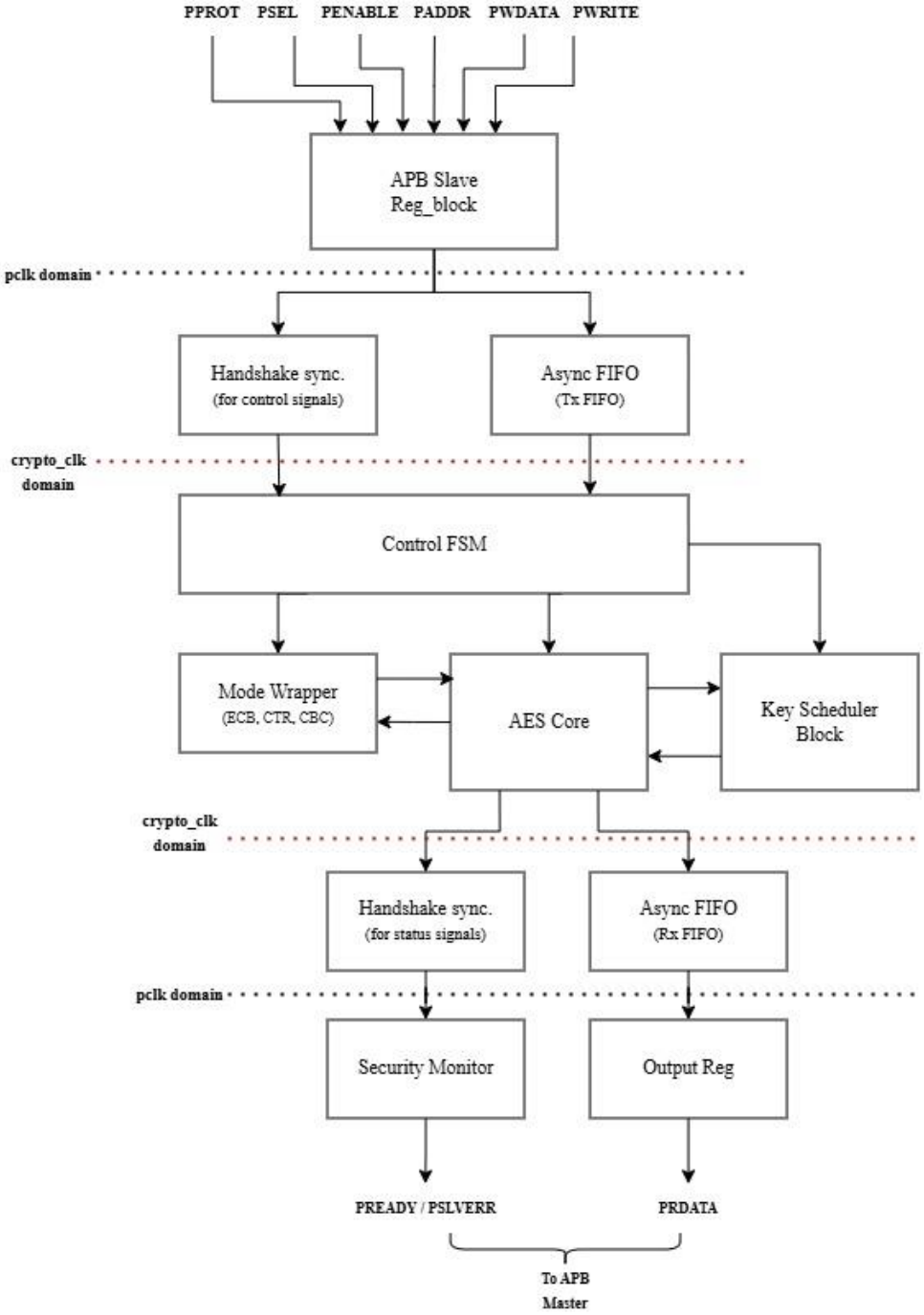


Figure 1.4 — Top-level block architecture

Chapter 2 — Signal Description

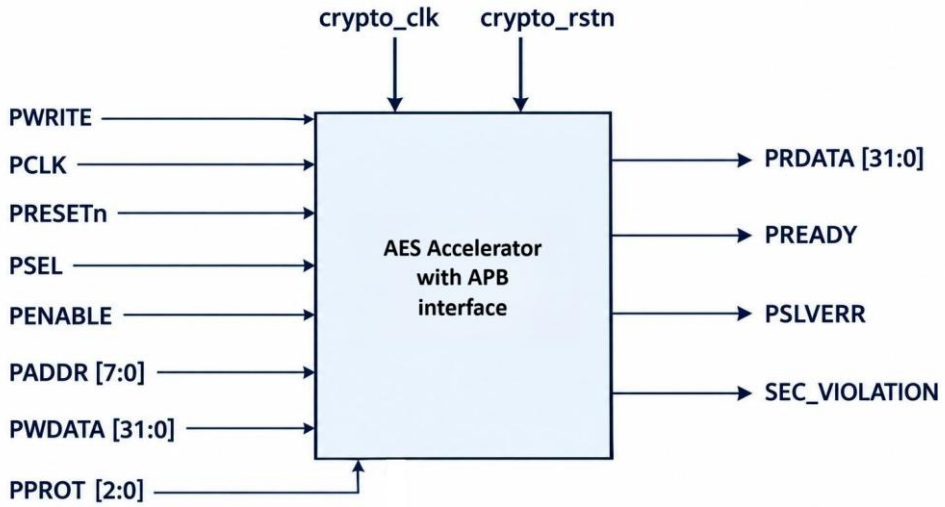


Figure 2.1 — Top-level port diagram

Type	Signal	Width	Description
Input	`PCLK`	1	APB interface clock
Input	`PRESETn`	1	Active-low APB reset (Asynchronous)
Input	`PSEL`	1	Selects the AES APB slave
Input	`PENABLE`	1	Indicates APB access phase
Input	`PWRITE`	1	1 = Write, 0 = Read
Input	`PADDR`	8	Register address, per the register map
Input	`PWDATA`	32	APB write data (key, block data, control signals)
Input	`PPROT[2:0]`	3	Protection attributes (normal/privileged, security, instruction/data). PPROT[0] = 1: privileged; PPROT[0] = 0: normal
Input	`crypto_clk`	1	Crypto clock (AES)
Input	`crypto_rstn`	1	Active-low crypto reset (Asynchronous)
Output	`PRDATA`	32	APB read data (ciphertext)
Output	`PREADY`	1	Indicates completion of the APB transfer (always high in our case)
Output	`PSLVERR`	1	Indicates an APB transfer error, if there is any unprivileged read/write happens.
Output	`SEC_VIOLATION`	1	Indicates a security-violation status, if there any violation happened in the current transaction. (Once `PSLVERR` = 1, the `SEC_VIOLATION` = 1 till the complete transaction (till reset)).

Chapter 3 — Register Map and Functional Description

3.1 APB Transfer Detection

A valid APB write is detected when all of the following hold:

- PSEL = 1
- PENABLE = 1
- PWRITE = 1

A valid APB read is detected when all of the following hold:

- PSEL = 1
- PENABLE = 1
- PWRITE = 0

3.2 APB Register Map

Address	Register	Purpose	Access Type
0x00	CONTROL	START, MODE, KEY_LOCK, ZEROIZE	Privileged
0x04	STATUS	BUSY, DONE	Normal/ Privileged
0x08	KEY0	32-bit key word	Privileged
0x0C	KEY1	32-bit key word	Privileged
0x10	KEY2	32-bit key word	Privileged
0x14	KEY3	32-bit key word	Privileged
0x18	DATA0	32-bit data word	Privileged
0x1C	DATA1	32-bit data word	Privileged
0x20	DATA2	32-bit data word	Privileged
0x24	DATA3	32-bit data word	Privileged
0x28	DATA_OUT0	Ciphertext	Normal/ Privileged
0x2C	DATA_OUT1	Ciphertext	Normal/ Privileged
0x30	DATA_OUT2	Ciphertext	Normal/ Privileged
0x34	DATA_OUT3	Ciphertext	Normal/ Privileged

3.3 CONTROL Register

The CONTROL register governs operation of the AES accelerator and has the following bit layout:

Bits [31:4]	Bit 3	Bit 2	Bit 1	Bit 0
Reserved	Zeroize	Key Lock	Mode	Start

Field behavior:

- Start — Setting Start = 1 begins AES processing. Before asserting Start, the master must ensure the key, data, and mode fields have all been written.
- Mode — Mode = 1 selects encryption; Mode = 0 selects decryption.
- Key Lock — Setting Key Lock = 1 locks all key registers and the mode register; they cannot be modified by software until the next reset.
- Zeroize — Setting Zeroize = 1 resets all datapath registers and APB registers to 0.

3.4 Key Loading

The AES key is 128 bits wide. Because each APB transfer carries 32 bits, the key is assembled from four sequential APB writes, in the following order:

Register	Key Bits	Write Order
KEY0	Key[127:96]	Word 1
KEY1	Key[95:64]	Word 2
KEY2	Key[63:32]	Word 3
KEY3	Key[31:0]	Word 4

The master must send the key words to the slave in this order.

3.5 Key Security

The key registers are write-only. Any APB read of a key register returns 0x00000000.

KEY_LOCK: once the key and mode have been received from the master, software can assert KEY_LOCK = 1. While KEY_LOCK = 1, all key registers and the mode register are locked and cannot be changed by software until the next reset.

3.6 Busy / Done Behavior

When an AES operation starts, BUSY = 1. The core remains busy until the complete block operation finishes. At completion, BUSY = 0 and DONE = 1. The DONE indication corresponds to the completed AES operation.

3.7 Privilege Checking

The APB protection attribute PPROT[0] is used for privilege checking:

- PPROT[0] = 1 — privileged access
- PPROT[0] = 0 — normal access

Security-sensitive accesses require the privileged attribute. For an unauthorized access, PSLVERR = 1 and SEC_VIOLATION = 1, and the unauthorized transaction does not modify any state.

During normal (non-privileged) access, the key, data, and mode registers cannot be changed, but PRDATA can still be read.

3.8 STATUS Register

The STATUS register provides non-sensitive operational information and has the following bit layout:

Bits [31:2]	Bit 1	Bit 0
Reserved	Busy	Done

Bit	Field	Description
1	BUSY	AES core busy
0	DONE	Operation complete

3.9 FSM State Transition:

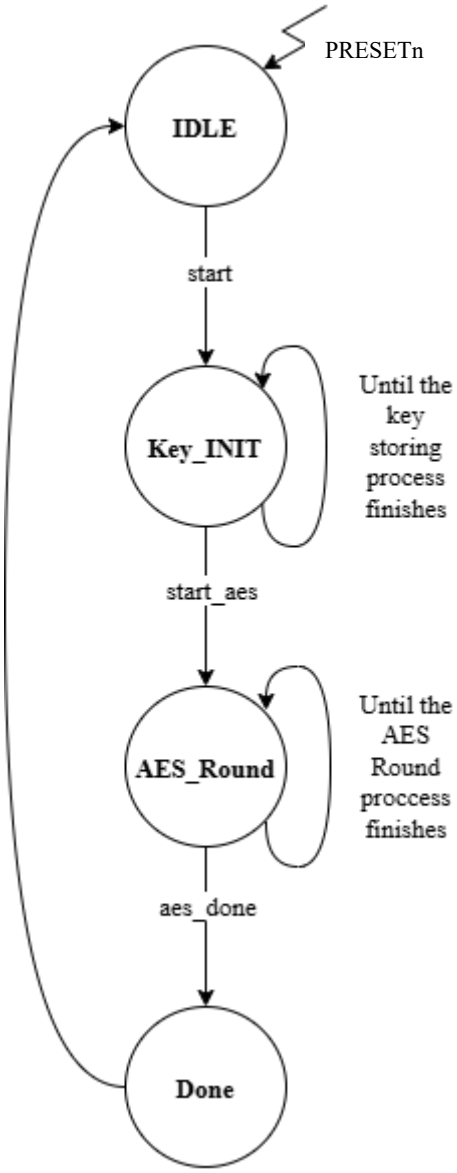


Figure 3.1 — FSM State diagram

Chapter 4 — Reset Behavior

On assertion of either reset — PRESETn = 0 (APB domain) and

On assertion of crypto_rstn = 0 (crypto-clock domain) — all sensitive information held by the core is cleared.

Both are Asynchronous Active low Reset signals.

Chapter 5 — Test Vectors

5.1 Reference Test Vectors (FIPS-197)

The verification environment uses the standard FIPS-197 test vectors below.

Encryption

Field	Value (hex, 4×32-bit words)
Plaintext	6BC1BEE2 2E409F96 E93D7E11 7393172A AE2D8A57 1E03AC9C 9EB76FAC 45AF8E51 30C81C46 A35CE411 E5FBC119 1A0A52EF F69F2445 DF4F9B17 AD2B417B E66C3710
Key	2B7E1516 28AED2A6 ABF71588 09CF4F3C
Ciphertext	3AD77BB4 0D7A3660 A89ECAF3 2466EF97 F5D3D585 03B9699D E785895A 96FDBAAF 43B1CD7F 598ECE23 881B00E3 ED030688 7B0C785E 27E8AD3F 82232071 04725DD4

Decryption

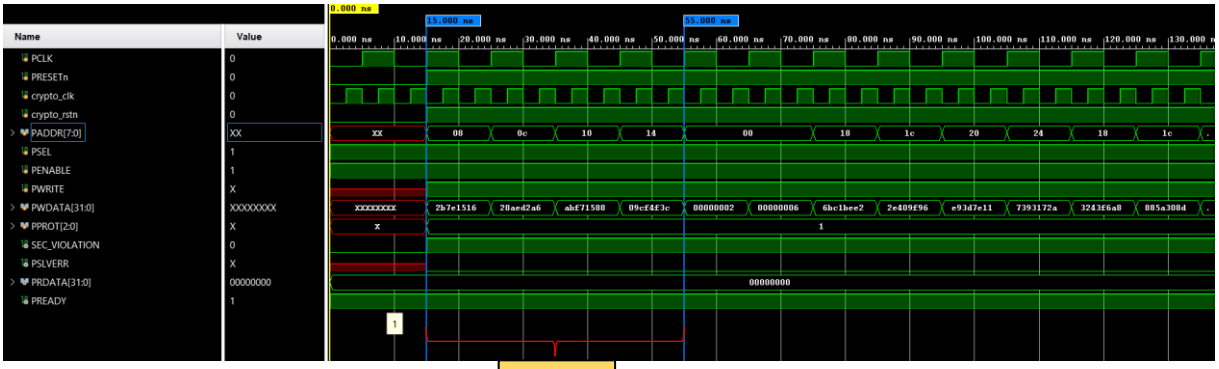
Field	Value (hex, 4×32-bit words)
Ciphertext	3AD77BB4 0D7A3660 A89ECAF3 2466EF97 F5D3D585 03B9699D E785895A 96FDBAAF 43B1CD7F 598ECE23 881B00E3 ED030688 7B0C785E 27E8AD3F 82232071 04725DD4
Key	2B7E1516 28AED2A6 ABF71588 09CF4F3C
Plaintext	6BC1BEE2 2E409F96 E93D7E11 7393172A AE2D8A57 1E03AC9C 9EB76FAC 45AF8E51 30C81C46 A35CE411 E5FBC119 1A0A52EF F69F2445 DF4F9B17 AD2B417B E66C3710

5.2 APB Write Transaction Sequence

A representative APB transaction sequence for an AES operation is:

1. Write key words 0–3 (KEY0–KEY3).
2. Write Mode and
3. Assert Key Lock.
4. Write data block words 0–3 (DATA0–DATA3). Repeat for each additional block.
5. Assert Start (Start may be de-asserted after one cycle).
6. Set PWRITE = 0 and read PRDATA for the corresponding PADDR, per the register map. Each output-block read requires one wait cycle before the next block word can be read.

Write Operation of Key (4 words, each 32 bits)



Write Key (4 words = 128 bit)

Figure 5.1 — Waveform: key write sequence

Write Operation of Mode and Key Lock

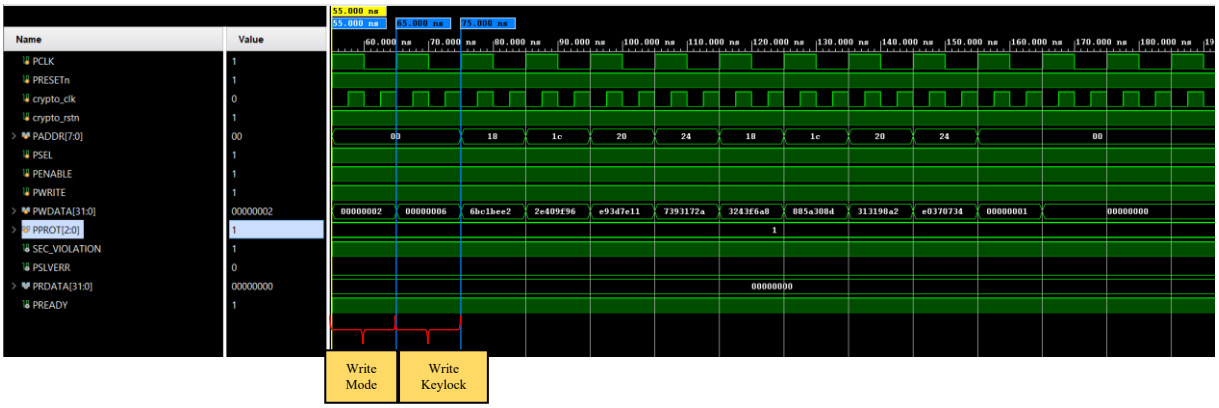


Figure 5.2 — Waveform: mode and key-lock write

Write of Block Data

Example data blocks used in this sequence:

- Data Block 1 = 6BC1BEE22E409F96E93D7E117393172A
- Data Block 2 = 3243F6A8885A308D313198A2E0370734

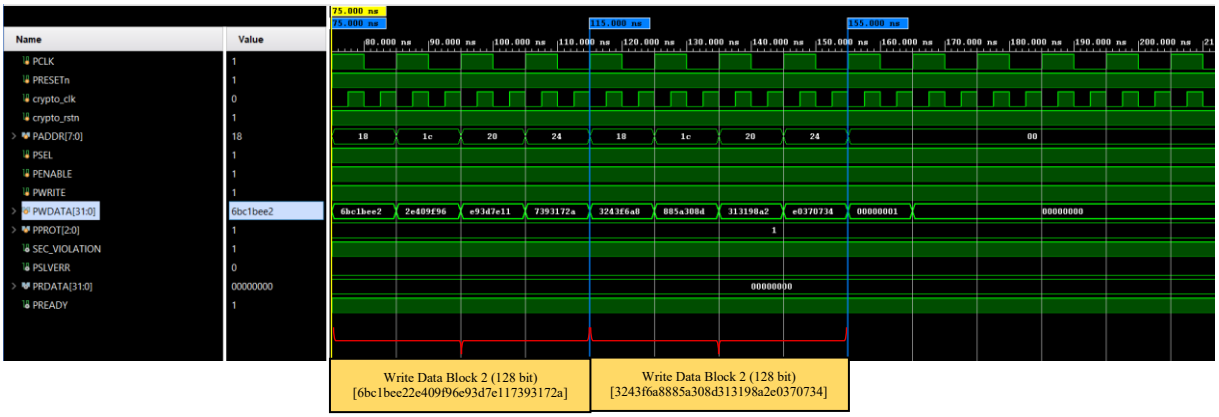


Figure 5.3 — Waveform: block-data write

Start of Process



Figure 5.4 — Waveform: Start assertion

Read Operation

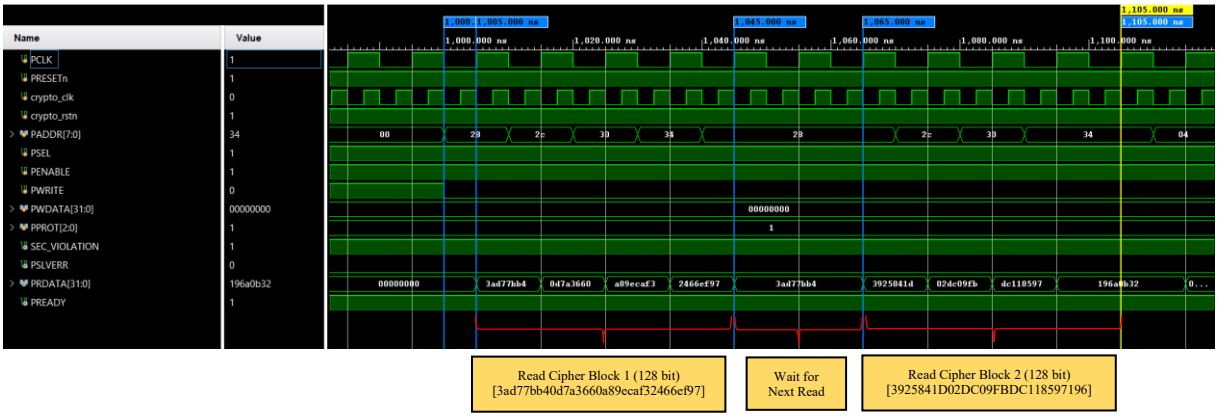


Figure 5.5 — Waveform: output read sequence